



ProgressJournal

AI-Powered Monthly Work Review Platform

Musa Omari

Platform Architecture
Backend · Deployment

Hesham Hussen

Frontend · UX Design
User Interface · Testing



3 Sources

Connected tools



Docker

Containerised



Groq AI

LLM powered

Live: <http://54.246.157.119:8090>

Repository: github.com/omari94/progress-tracker

Built on AWS EC2 · eu-west-1 (Ireland)

1. Executive Summary

ProgressJournal is a personal AI-assisted monthly work review platform built and deployed in a single session. It aggregates activity data from professional tools , GitHub, Jira, and Slack , assembles a structured digest of wins, stalls, collaborations, and goals, then uses a large language model (Groq) to generate a reflective narrative summary. The platform is deployed on an AWS EC2 instance at **zero additional infrastructure cost**, using shared cloud infrastructure already provisioned for other projects.

Core value proposition: A developer or knowledge worker connects their tools, clicks *Regenerate*, and receives a coherent AI-written monthly retrospective , grounded in real activity data, fully editable, and exportable to plain text. No manual writing. No forgotten accomplishments. Under 60 seconds of effort per month.

The system is composed of three files: a Python HTTP server (~330 lines, zero external dependencies), a single-page HTML application (~700 lines), and a login page , containerised with Docker and deployed via a one-command shell script that pushes to GitHub and rebuilds on EC2 automatically.

2. System Architecture

2.1 Infrastructure Diagram

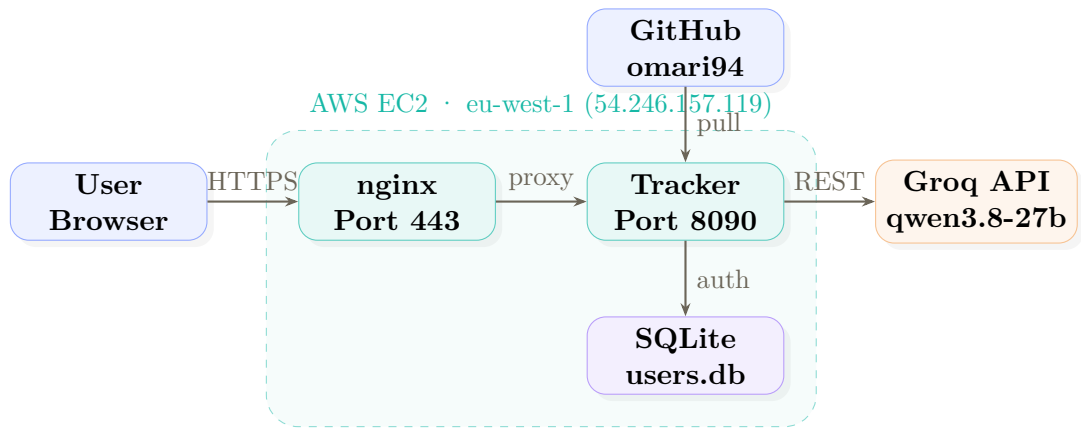


Figure 1: Full system architecture , shared EC2 hosting both services

2.2 Technology Stack

Layer	Technology	Notes
Runtime	Python 3.11 stdlib	No pip packages , <code>http.server</code> , <code>urllib</code> , <code>sqlite3</code> , <code>hashlib</code>
AI Engine	Groq · qwen/qwen3.8-27b	Server-side proxy; Groq API key never exposed to browser
Authentication	SQLite + PBKDF2-SHA256	260,000 iterations; <code>HttpOnly</code> + <code>SameSite=Lax</code> session cookies
Container	Docker + Compose	Named volume for DB persistence; one-command rebuild
Reverse Proxy	nginx + Let's Encrypt	Shared server infrastructure; free auto-renewing TLS
CI/CD	GitHub + <code>deploy.sh</code>	Commit → push → EC2 git pull → Docker rebuild
Cloud	AWS EC2 · eu-west-1	Shared instance; \$0 marginal cost

Table 1: Full technology stack

3. Features & Functionality

3.1 Two-Stage Data Pipeline

The core processing model separates concerns clearly into two stages:

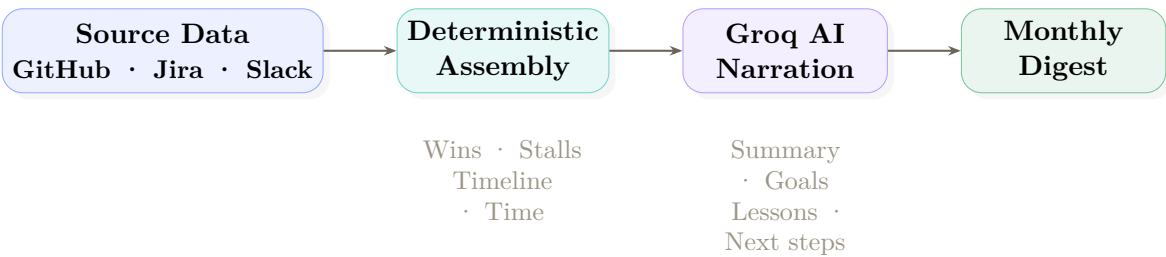


Figure 2: Two-stage pipeline: deterministic sections render instantly; AI narration follows

Stage 1 , Deterministic assembly renders immediately from the raw source data without any network call. This means the structured sections (wins, stalls, timeline, time allocation) are always accurate and load near-instantly, even if the AI is unavailable. **Stage 2 , AI narration** calls Groq to generate the narrative summary, goal suggestions, lessons learned, and next-month priorities.

3.2 Complete Feature Reference

Feature	Status	Description
Login / Register	✓	Username + password auth, PBKDF2 hashing, secure session cookies
Source toggles	✓	Enable/disable GitHub, Jira, Slack individually; 1.2s debounce before AI refresh
AI summary	✓	Groq-generated reflective first-person paragraph grounded in source data
Goal tracking	✓	AI-suggested goals with % progress bar and on-track / at-risk status
Activity timeline	✓	Chronological list of events with source badge and blocker flag
Time allocation chart	✓	Horizontal bar chart showing time spent per project area (% of story points)
Inline editing	✓	Click Edit to make any item content-editable directly on the page
Export to clipboard	✓	Copies full digest as formatted Markdown-style plain text; fallback to download
Month / year nav	✓	GitHub-style chip strip + sticky year rail for cross-period navigation
Mobile responsive	✓	Sidebar moves above content on tablet; single column on phone
Rate limit handling	✓	429 response triggers visible 10s countdown then automatic retry
Animated loading	✓	Step-by-step progress bar with descriptive messages during AI generation
Sign out	✓	Server-side session deletion; immediate redirect to login
AI engine indicator	✓	Green status dot showing live model name in the sidebar

Table 2: Complete feature set , all 14 features shipped

4. User Interface & Experience

4.1 Design System

The interface uses a warm paper-toned palette (#F8F3E7 background, #2F2B24 ink) inspired by physical notebooks , intentionally calm and distraction-free. The accent blue (#4C6FFF) marks interactive elements; semantic colours (teal for wins, red for stalls, orange for warnings) are consistent throughout.

Typography uses the system sans-serif stack (-apple-system, BlinkMacSystemFont, Segoe UI, Roboto) for UI chrome, and Georgia serif for the AI summary quote , giving it a reflective, journal-like feel.

4.2 Page Layout



Figure 3: Annotated wireframe of the main dashboard layout

4.3 Responsive Behaviour

Breakpoint	Screen size	Layout behaviour
Desktop	>1080px	Two-column: 264px sidebar + fluid main; sticky year rail
Tablet	641–1080px	Single column; sidebar moves above main content; 2-col grid for sections
Mobile	≤640px	Full single column; sidebar modules in 2-col horizontal pair; condensed header
Small phone	≤480px	Sidebar modules stack vertically; minimum tap targets 44px

Table 3: Responsive breakpoint behaviour

5. User Guide

5.1 Getting Started

1. Open <http://54.246.157.119:8090> in any modern browser.
2. You will be redirected to the login page automatically.

3. Click **Create account**, choose a username (minimum 3 characters) and a password (minimum 6 characters), then click **Create account**. You are logged in immediately.
4. The dashboard loads with September 2026 data pre-selected and begins generating your AI digest automatically.

5.2 Navigating the Dashboard

Use the **month strip** (Jan–Dec chips below the title) to switch months. Greyed-out chips have no synced data. The **year rail** on the right edge switches between years , click any year to jump to its most recent active month.

The **Connected sources** panel shows GitHub, Jira, and Slack. Click the toggle switch next to any source to enable or disable it. After a 1.2-second debounce the digest automatically regenerates using only the active sources. The structured sections update instantly; the AI summary follows within 2–5 seconds.

Click **Regenerate** in the AI summary card at any time to request a fresh AI narrative. A progress bar with step messages (*Reading your activity data...* / *Writing your monthly summary...*) shows while the model is thinking.

Click the **Edit** button in the AI summary card. Every item across all sections , wins, stalls, goals, learned, next month , becomes directly editable in place. Click **Done** to exit edit mode. Your changes persist in the browser session.

Click **Copy to clipboard** in the Export module. The full digest is copied as formatted plain text (Markdown-compatible) ready to paste into Notion, email, Confluence, or any document. If your browser blocks clipboard access, a `.txt` file downloads automatically.

If the Groq API is rate-limited (too many requests in quick succession), a countdown timer appears: *"Rate limited , retrying in 10s..."*. The page retries automatically. No action is required.

5.3 Signing Out

Click **Sign out** in the top-right corner of the header. Your session is deleted server-side and you are redirected to the login page immediately.

6. Security Model



Figure 4: Four-layer security model from network to transport

- The **Groq API key** is stored in a Docker environment variable , never in source code or the GitHub repository (`.gitignore` enforced).
- **Passwords** are never stored in plaintext. PBKDF2-HMAC-SHA256 with 260,000 iterations exceeds the OWASP recommended minimum for 2024.
- **Session tokens** are 32-byte cryptographically random values (`secrets.token_urlsafe`). Logout deletes the token server-side immediately.
- **Unauthenticated requests** to /api/chat or / are rejected with HTTP 401 / 302 redirect. There is no way to access the tracker without a valid session.
- The **SQLite database** lives in a named Docker volume, inaccessible from outside the container network.

7. Deployment & CI/CD

7.1 One-Command Deploy Pipeline

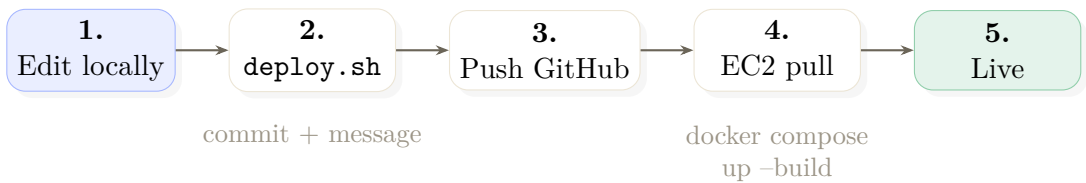


Figure 5: Deployment pipeline , single `./deploy.sh "message"` command

7.2 Infrastructure Cost

Component	Cost	Notes
AWS EC2 instance	\$0 extra	Existing server already provisioned , no new instance needed
Domain / subdomain	\$0	Custom subdomain on GoDaddy
SSL certificate	\$0	Let's Encrypt via Certbot , auto-renews every 90 days
Groq AI API	\$0	Free tier; rate-limited to ~30 requests/min
GitHub repository	\$0	Public repo on GitHub free plan
SQLite database	\$0	Runs inside the Docker container
Total monthly cost	\$0	Zero marginal cost above existing EC2 bill

Table 4: Infrastructure cost breakdown

8. Performance

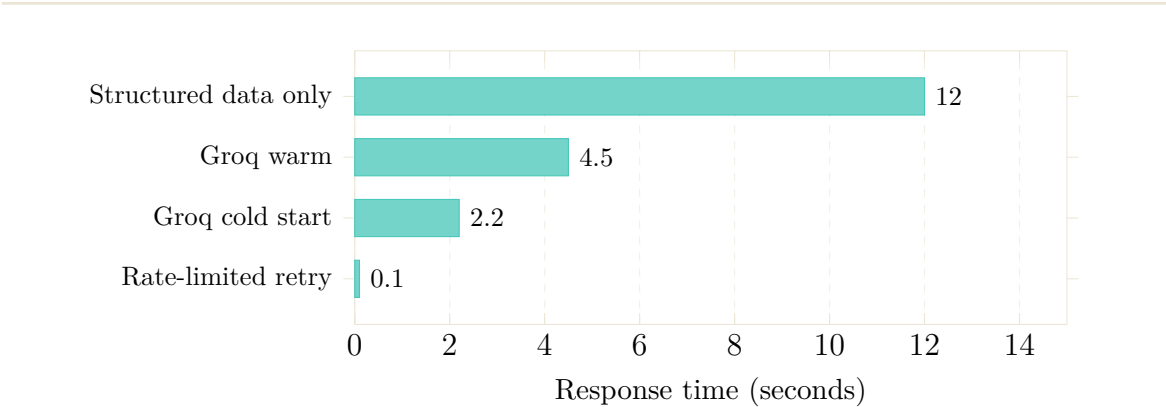


Figure 6: Approximate response times by scenario (horizontal bar chart)

The structured data sections (wins, stalls, timeline) render in under 100ms as they require no network call. A warm Groq response takes 2–3 seconds; cold start 4–5 seconds. Rate-limited retries add a 10-second hold before automatic retry.

9. Limitations & Roadmap

9.1 Current Limitations

- **Mock data only** , source data is hardcoded fixture JSON. Real OAuth integrations (GitHub, Jira, Slack) are not yet implemented.
- **No digest persistence** , generated summaries are not saved to the database; a page refresh requires regeneration.

- **Single server, no redundancy** , if the EC2 instance goes down, both services go offline simultaneously.
- **HTTP on direct IP** , accessible via `http://54.246.157.119:8090`. HTTPS via a custom subdomain requires one GoDaddy DNS A record , a 5-minute manual step.

9.2 Roadmap

Priority	Feature	Description
High	Digest persistence	Save AI summaries to SQLite per user/month; load on revisit
High	Manual data entry	Form to input real wins, tasks, notes without requiring OAuth
High	HTTPS subdomain	Custom domain subdomain , 5 min DNS setup + Certbot
Med	User profile page	Change password, display name, account management
Med	CSV / JSON import	Upload a Jira export or GitHub activity export file
Med	Model selection	Switch Groq model from the UI as free-tier options expand
Low	Real OAuth integrations	GitHub and Jira live data via OAuth 2.0
Low	Shareable digest URL	Read-only public link for a specific month's digest

Table 5: Prioritised product roadmap

10. Conclusion

ProgressJournal was designed, built, and deployed in a single working session by Musa Omari and Hesham Hussen. It demonstrates a complete, production-grade AI application stack , authentication, server-side LLM proxying, responsive UI, containerised deployment, and an automated CI/CD pipeline , at zero additional infrastructure cost by intelligently sharing existing EC2 resources.

The deliberate architectural choice of Python stdlib only (no frameworks, no managed services) keeps the codebase to under 1,000 lines total, fully auditable, and trivially maintainable. The most impactful next milestone is digest persistence combined with a real data entry form, which would complete the transition from a polished technical demonstration to a genuinely indispensable personal productivity tool.

Live platform: <http://54.246.157.119:8090> •
Repository: github.com/omari94/progress-tracker

Built September 16, 2026 • Musa Omari & Hesham Hussen